

BPI-CM6 rc3 完整測試與驗證手冊

版次：2026-09-23 | Ubuntu 24.04 Noble／GNOME | 用途：候選驗證，不是發布合格證明。

一頁快速順序

每張 SD、每次 eMMC 燒錄各建立一份獨立紀錄；按照下列順序執行，細節見後文相同測試 ID。任何必要項目未測，填 **BLOCKED**，不可填 PASS。

順序	測試 ID	操作與關卡
1	CM6-001～002	拍載板版本、接線與散熱；確認可覆寫媒體；核對 ZIP 與解壓後 SHA。
2	CM6-003 或 004	SD 寫整碟 .img；eMMC 交給 Titan 匯入專用 ZIP。兩種檔案不可互換。
3	CM6-005～007、013	上電前開始 UART 接收；記錄乾淨首開藍牙，再做登入、身分、分區及 UUID 檢查。
4	CM6-008～016	桌面、USB、eth0、eth1、Wi-Fi、藍牙及音訊；每個介面獨立留證。
5	CM6-017～022	核對兩顆 NVMe 身分後先唯讀，再測明確准寫的既有檔案系統；GPU 像素比對、AI CPU 對照與實際後端追蹤。
6	CM6-023～027	先測原 rc3 相機配置；另開候選批次測兩路 IMX415、雙路停止、立即重開與畫質。
7	CM6-028～029	全程記溫度與降頻；散熱合格後才做 30 秒短壓測。
8	CM6-030～033	暖重啟一次、正常關機後斷電至少五秒再開三次、關機與復原演練。
9	CM6-034	保存原始資料、產生 SHA 清單、填 CSV 與缺陷報告；逐項簽核，候選與原版分開。

立即停止條件：目標磁碟不明、載板接腳或開機選擇不明、供電不符、相機排線未斷電插拔，均停止相關操作。溫度達 90°C 停止新增負載；負載前須低於 75°C 且無降頻。這是本手冊的保守測試門檻，不是晶片額定值。

本批已知未過：藍牙新卡首開軟體封鎖；Wi-Fi 接收丟棄計數增加；HDMI 青色畫面與 EDID 失敗；受測 AI 模型回退 CPU；雙相機停止逾時、畫質及 30 FPS 未驗收；SoC 曾達 92～94°C 並降頻。rc3 eMMC 整包仍須重新實測。測試不得掩蓋這些結果。

結果碼：PASS＝本項全部條件及證據齊全；FAIL＝已執行而不符條件；BLOCKED＝缺設備、工具、前置條件或證據；NA＝經審核確定該媒體／硬體不適用，須寫理由。空白不是結果。

1. 工作位置、證據規則與硬體準備

CM6-001：建案與接線

本手冊只適用 BPI-CM6 的 20260923-rc3。K1 是 RISC-V 平台。不得將 F3、CM4、CM5 或 BPI-M6 的接腳、工具或映像套用到 CM6。既有 Armbian 原始格式與本官方格式相容候選是不同流程。

pi-88 的交付根目錄如下。離線測試人員應取得兩個媒體目錄及共同的 驗證文件，保持相對位置。

```
/media/pi/SMCI/bpi/f3-cm6-vendor-20260923-rc3/output/  
  bpi-cm6-sd/          ← SD ZIP、manifest.json、verification.json、SHA  
  bpi-cm6-emmc/        ← Titan ZIP、manifest.json、verification.json、SHA  
  驗證文件/           ← 本手冊 MD/PDF、CSV、驗證工具、SHA256SUMS
```

準備下列設備，記入主機案件的 host/CM6-001-hardware.md，拍攝 photos/CM6-001-carrier.jpg、photos/CM6-001-wiring.jpg。照片不得包含網路密碼、帳號標籤或敏感畫面。

- CM6 模組序號／版本、RAM 容量、eMMC 容量；載板完整型號與 PCB revision；實物上的開機開關位置照片。參考原理圖是 **V00**，不代表目前待測載板已確認為 V00。
- 符合該載板官方規格的電源、額定電壓／電流、實測供電方式；散熱片、導熱材料、風扇與室溫。先確認供電，不猜電源口規格。
- 可覆寫且容量足夠的 SD、讀卡器、USB 資料線、Linux 測試主機；Titan 可用 Windows 或 Linux，記錄 OS、實際版本及安裝檔 SHA。
- 3.3 V TTL UART，115200 baud、8 資料位、無同位、1 停止位、無流控。**板 TX 接轉接器 RX、板 RX 接 TX、共地；不接 VCC，也不接 RS-232 電平。**接腳依對應 revision 原理圖核對。
- 已知良好 HDMI 螢幕與線、擷取器（若有）、USB 鍵鼠、既有檔案系統的測試隨身碟；GbE 交換器與兩條線、隔離測試 AP、可配對的藍牙測試設備、喇叭／耳機與麥克風。
- 128 GB 與 256 GB NVMe 的型號、序號及准許寫入範圍；兩顆相容 IMX415、正確排線／轉接板與均勻光源、解析度／色彩測試圖。**相機排線只在完全斷電時插拔。**

本機歷史接法為 UART /dev/ttyUSB0、HID /dev/ttyUSB1、電源 bpi-pw-2，只作定位線索；每次須按 USB 身分重新核對，不能靠枚舉號自動操作。擷取主機的 /dev/video0 與 CM6 的 /dev/video0 是不同設備；後者曾是編解碼器，不能直接當 CSI 相機。

主機工具檢查：Bash、Python 3、pyserial、unzip、sha256sum、lsblk、Etcher、Titan；收集與後端工具的缺失按個別測項列 BLOCKED，不先全面升級候選系統。工具包含 GPU 測試、AI 固定模型及相機配套；應從本次交付取得，不任意抓浮動版本。

```
python3 -c 'import serial; print(serial.VERSION)'  
command -v python3 unzip sha256sum lsblk
```

PASS：型號、revision、容量、接腳、供電與准寫範圍皆明確且有照片。缺一必要資訊則相關測項 BLOCKED；不得用推測補欄位。

共用記錄方法

【主機】在一般使用者終端機建立案件；以下變數需在各自的終端機重新設定。CASE 用匿名板號，勿用帳號或密碼。SD／eMMC 及每次重燒各用新案件後綴；原版與相機候選用本手冊不同測項子目錄分開，CSV 備註明確標記配置。

```
umask 077
read -r -p '案件代號 (例如 CM6-A01-sd-rc3-R01) : ' CASE
[[ "$CASE" =~ ^[A-Za-z0-9_-]+$ ]] || exit 1
E="$HOME/cm6-evidence/$CASE"
mkdir -m 700 -p "$HOME/cm6-evidence"
mkdir -m 700 "$E" || exit 1
mkdir "$E/host" "$E/board" "$E/photos" "$E/defects"
DOC="/media/pi/SMCI/bpi/f3-cm6-vendor-20260923-rc3/output/驗證文件"
cp "$DOC/BPI-CM6-rc3-測試結果表-20260923.csv" "$E/results.csv"
```

離開 pi-88 時將 doc 改成下載後的共同文件目錄。以安全管道取得首次登入資料，**本手冊、命令列、CSV、截圖、UART 與日誌均不得寫入密碼或金鑰**。不要執行 `set -x`、`nmcli --show-secrets`、列印環境變數，或複製 NetworkManager 連線設定／私鑰。登入及輸入密碼時不開終端錄影；必要時先結束 UART 接收並記錄停錄時段。

【板端】完成帳號建立後，把 驗證工具 整個目錄以現場核實的 SSH 或已知 USB 媒體傳到 `$HOME/cm6-tools`。不提供內嵌帳密的遠端命令。輸入相同 CASE，建立獨立板端目錄：

```
umask 077
read -r -p '案件代號: ' CASE
[[ "$CASE" =~ ^[A-Za-z0-9_-]+$ ]] || exit 1
E="$HOME/cm6-evidence/$CASE/board"
mkdir -m 700 -p "$HOME/cm6-evidence/$CASE"
mkdir -m 700 "$E" || exit 1
T="$HOME/cm6-tools"
(cd "$T" && sha256sum -c SHA256SUMS) > "$E/tools-sha256.txt" 2>&1
```

板端一般使用者先執行 `export PATH="/usr/sbin:$PATH"`，確保找到診斷命令。每次測試用新證據檔；命令若失敗保留原檔，重測命名加 -R02，不得覆蓋。下文 `|& tee` 需在 Bash 執行；**先 `set -o pipefail`**，以免 `tee` 掩蓋失敗。每組命令把真實退出碼、開始／結束時間、人工觀察寫入該測項記錄。sudo 驗證先在未錄製的終端完成。

共用收集器：

```
sudo bash "$T/collect_cm6_rc3_validation.sh" --output "$E/baseline"
```

以 sudo 執行的收集器／相機工具會建立 root 所有的私有目錄。每次完成後，先確認 `$E` 是本案件的 board 證據目錄，再只調整這個目錄的所有權，讓一般使用者可讀及傳檔；不可對 `/`、`/root` 或整個家目錄執行：

```
realpath "$E"
test "$E" = "$HOME/cm6-evidence/$CASE/board" || exit 1
test -d "$E" && test ! -L "$E" || exit 1
sudo chown -R --no-dereference -- "$(id -u):$(id -g)" "$E"
```

每次重開／重新登入後，shell 變數不會保留。**不要重新建立已存在的案件**，使用以下捷徑重設路徑：

```
read -r -p '既有案件代號: ' CASE
[[ "$CASE" =~ ^[A-Za-z0-9_-]+$ ]] || exit 1
E="$HOME/cm6-evidence/$CASE/board"
T="$HOME/cm6-tools"
CAMROOT="$HOME/cm6-camera-$CASE"
ORIG="$E/CM6-024-extlinux.original.conf"
export PATH="/usr/sbin:$PATH"
test -d "$E" || exit 1
```

它只收集，不啟用裝置，也不把退出碼零當硬體 PASS。每個命令有獨立 stdout/stderr 與 command-status.tsv；缺工具或逾時有明確狀態。必須檢查表內失敗，不能只看收集器退出成功。

2. 成品、容量與完整性

CM6-002：核對檔案

所有大小均為精確 **bytes**，不是磁碟廠商標稱 GB。兩種媒體 minimum_media_bytes 都是 **6,716,129,280**；實際媒體必須不小於此值，建議準備 16 GB 以上。主機應保留至少 20 GB 空間，證據照片／錄影另計。

SD 壓縮檔

```
Armbian_Noble_bpi-cm6_gnome_vendor-sd_20260923-rc3.img.zip
大小：1713117447
SHA-256：228cef7811e763cbf0b8f0d57d55ba78f79f87f2f3bf5e7f59d5d53bd94c78a7
```

SD 解壓後整碟映像

```
Armbian_Noble_bpi-cm6_gnome_vendor-sd_20260923-rc3.img
大小：6716129280
SHA-256：562cd7820b788df1a35e4763a23d585262684015c0fbb2921fc1fa23c0608766
```

Titan eMMC 套件

```
Armbian_Noble_bpi-cm6_gnome_titan-emmc_20260923-rc3.zip
大小：1713113819
SHA-256：06e02aa4bfb309535e7858add46d20745f5fbf65f94ccfd21e4acf974e2d55a3
```

SD ZIP 僅含一份整碟 GPT .img。Titan ZIP 是十個分區／載荷及設定成員，含 fastboot.yaml 與 partition_universal.json；它不是整碟映像，不能交給 Etcher/dd，也不能改副檔名互換。bootinfo_sd.bin 等名稱屬固定官方協定配置，不自行換成看似更合適的成員。

【主機】設定交付根目錄後核對；下面只讀檔案，不會燒錄：

```
OUT='/media/pi/SMCI/bpi/f3-cm6-vendor-20260923-rc3/output'
SD='Armbian_Noble_bpi-cm6_gnome_vendor-sd_20260923-rc3.img'
EM='Armbian_Noble_bpi-cm6_gnome_titan-emmc_20260923-rc3.zip'
set -o pipefail
{
  stat -c '%n %s' "$OUT/bpi-cm6-sd/$SD.zip" "$OUT/bpi-cm6-emmc/$EM"
  (cd "$OUT/bpi-cm6-sd" && sha256sum -c "$SD.zip.sha256")
  (cd "$OUT/bpi-cm6-emmc" && sha256sum -c "$EM.sha256")
  unzip -t "$OUT/bpi-cm6-sd/$SD.zip"
  unzip -t "$OUT/bpi-cm6-emmc/$EM"
} |& tee "$E/host/CM6-002-archives.txt"
mkdir "$E/host/sd-unpacked" || exit 1
unzip -n "$OUT/bpi-cm6-sd/$SD.zip" -d "$E/host/sd-unpacked"
stat -c '%n %s' "$E/host/sd-unpacked/$SD"
sha256sum "$E/host/sd-unpacked/$SD" | tee "$E/host/CM6-002-image.sha256"
```

也可在 Windows 用 `Get-FileHash -Algorithm SHA256 -LiteralPath <實際 ZIP 路徑>`，大小用 `(Get-Item -LiteralPath <實際路徑>).Length`。尖括號不是可直接貼上的參數，必須換成實際路徑。Titan 可自行解包；人工查看時解至新目錄，不重壓或修改套件。

PASS：上述大小、三個 SHA 與本頁固定值完全相符，ZIP 檢查無錯。每一行均須核對；某行成功不代表整組成功。

FAIL：任一不符，停止燒錄並重新取得成品。證據：`host/CM6-002-archives.txt`、`host/CM6-002-image.sha256`，另保存兩個 manifest 與 verification 到各自的 `host/sd-metadata/`、`host/emmc-metadata/`，不可互相覆蓋。

3. 燒錄：只執行本案件選定的一種媒體

CM6-003：SD 整碟燒錄

破壞性警告：燒錄會覆寫選定磁碟的分區及全部資料。不得選主機系統碟、備份碟、未知 USB 碟或單一分區。本手冊刻意不提供自動找卡後執行 `dd` 的腳本。

1. SD 插入前及插入後，分別執行 `lsblk -b -o NAME,PATH,TRAN,MODEL,SERIAL,SIZE,TYPE,MOUNTPOINTS`，保存為 `host/CM6-003-before.txt`、`host/CM6-003-after.txt`。按新增裝置、卡片實物／容量／序號共同核對；只看 `/dev/sdX` 名稱不夠。
2. 確認可覆寫授權及資料備份；卸載已辨識目標的掛載分區。不可執行萬用字元卸載或自動取第一顆磁碟。
3. 開啟 Etcher，記錄版本。在 Flash from file 選 CM6-002 已核對的 `.img`；在 Select target 核對磁碟容量與實物；再按 Flash!。讓工具完成寫入後驗證，保存 `photos/CM6-003-flash.png`。
4. 彈出卡片，再插回讀卡器但不要開機；人工指定剛才同一張卡的整碟路徑，唯讀回讀前 6,716,129,280 bytes。此處不會寫磁碟：

```
read -r -p '已核對的 SD 整碟路徑：' CARD
lsblk -b -o NAME,PATH,MODEL,SERIAL,SIZE,TYPE,MOUNTPOINTS "$CARD"
sudo blockdev --getsize64 "$CARD"
# 只有容量與實物確認一致才繼續；以 pipefail 檢查讀取是否成功。
set -o pipefail
sudo head -c 6716129280 "$CARD" | sha256sum \
  | tee "$E/host/CM6-003-readback.sha256"
```

PASS：Etcher 寫入／驗證皆成功，回讀 SHA 等於 SD 原始映像 SHA。開機後會擴容及寫檔，不得再要求整卡雜湊仍等於原始檔。**FAIL：**回讀短讀、I/O 錯誤或 SHA 不同。證據含目標識別與 `host/CM6-003-readback.sha256`。eMMC 案件此項填 NA。

CM6-004：Titan eMMC 燒錄

破壞性警告：Titan 會改寫 eMMC；本候選並未在包內硬鎖目標媒體。先正常關機、斷電，拔除 SD、NVMe 及其他可寫入儲存裝置。核對載板 revision 的 eMMC 開機選擇；不確定則 BLOCKED，不猜 DIP／跳線、不選 NOR／NAND／MTD 設定。

1. 從 [CM6 官方入門頁](#) 進入 Titan 工具下載。2026-09-16 留存入口是 1.0.35-beta，尚無 rc3 的 Titan 相容版本已驗證清單。記錄工具實際版本、安裝檔名／SHA、主機 OS、驅動版本於 `host/CM6-004-titan-version.txt`；不只填「最新版」。
2. 主機僅連一片待測 CM6；用可傳資料的 USB 線直連載板指定下載口。按實物／官方文件核對 DOWNLOAD/FEL 按鍵：斷電時按住再接電，或按住後觸發重置，再依官方流程放開。不要把電源口當成下載口，不套用 F3 的按鍵圖。
3. Titan 單機模式確認被識別的板子，保存 VID:PID／序號及 `photos/CM6-004-detected.png`。無識別先查線、供電、下載模式與驅動；不可猜測改 EEPROM。

4. 匯入 `Armbian_Noble_bpi-cm6_gnome_titan-emmc_20260923-rc3.zip`，等待解包及配置載入。再次確認目標後開始燒錄；保存從連線到結束的原始日誌為 `host/CM6-004-titan.log`，完成畫面為 `photos/CM6-004-result.png`。
5. 完成後完全斷電，退出下載狀態，保持 SD、NVMe 已移除，再按下一節從上電前錄 UART；確認 eMMC 獨立開機與本批 UUID。不要以「工具成功」代替啟動驗證。

PASS： Titan 全流程成功+正確 rc3 eMMC 身分獨立冷開成功。**FAIL：** 匯入、分區匹配、傳輸或啟動任一步錯誤。Code 20014 需保留原始日誌，不改 ZIP、不刪元件、不自行寫 boot0。SD 案件填 NA。rc2 使用者曾回報 Titan 燒錄啟動成功，**不能代替本批 rc3 結果。**

4. 上電、首開與系統基線

CM6-005：上電前 UART、冷開與首次設定

【主機】只接已核對 UART，確認沒有另一個終端程式占用；HID 串口不得使用。接收器只讀、不會送登入資料。pyserial 缺少則先在測試主機補足，無 UART 證據的本項列 BLOCKED。

```
read -r -p '已核對的 UART 路徑：' UART
python3 "$DOC/驗證工具/cm6_rc3_uart_capture.py" \
  --port "$UART" --output "$E/host/CM6-005-first-power uart.bin" \
  |& tee "$E/host/CM6-005-capture.txt"
```

看到「接收已就緒」才上電，在另一份 `host/CM6-005-timeline.txt` 記錄 UTC 上電時間、電源指示、首個 UART 字元、核心啟動、登入提示及可登入時間。最多等五分鐘；無進度或 panic 記 FAIL，先留證再復原。第一次可能有檔案系統初始化與設定，不中途拔電。

到登入提示後 Ctrl+C 結束接收，透過未錄製的主控台完成首次帳號、語系與時區設定；不得把登入資料存進證據。記錄停錄起迄，再開新的 `host/CM6-005-after-setup uart.bin` 接收。不要立即解除 rfkill 或改藍牙，先執行收集器並保存下面的首開狀態，供 CM6-013 判讀：

```
{ date -u --iso-8601=seconds; rfkill list; bluetoothctl list;
  bluetoothctl show; systemctl --no-pager status bpi-cm6-bluetooth.service;
} > "$E/CM6-013-fresh-bluetooth.txt" 2>&1
```

PASS： 從上電到登入前的原始 UART 完整、無 panic／反覆重啟、五分鐘內可完成啟動；首次設定成功，停錄範圍明確。密碼輸入的停錄不是缺陷，但不能剪去啟動錯誤。時間超限須保留原因；有服務可登入不等整機所有項目 PASS。

CM6-006：版本與啟動身分

【板端】

```
{ date -u --iso-8601=seconds; uname -a; cat /etc/os-release;
  cat /etc/armbian-release; cat /etc/bpi-k1-vendor.json;
  cat /proc/device-tree/model; printf '\n'; cat /proc/sys/kernel/tainted;
  dpkg-query -W bpi-cm6-bluetooth; systemctl --failed --no-pager;
} > "$E/CM6-006-identity.txt" 2>&1
```

預期 Ubuntu 24.04、board=bpi-cm6、release_id=20260923-rc3、符合本案件的 storage；核心 6.6.36-legacy-spacemit、藍牙套件 0.1.0~20260923rc3、taint 0。記錄工具包 SHA 與 manifest。**PASS：** 全部相符且無未解釋的失敗單元；任一錯板／錯媒體／錯版本 FAIL，停止後端效能判定。

CM6-007：分區、檔案系統與擴容

```
{ findmnt /; findmnt /boot;
  lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,PARTUUID,MOUNTPOINTS;
  df -B1 / /boot; cat /proc/cmdline;
  sudo journalctl -b -u bpi-k1-grow-rootfs.service --no-pager;
  sudo dmesg --color=never;
} > "$E/CM6-007-storage.txt" 2>&1
```

媒體	rootfs UUID	bootfs UUID
SD	b420f97c- df24-5531-974c-582f4a977d96	2b3957a6-2286-5464- ba31-8f22284cb925
eMMC	88c6221a-58a5-5fd9-b74e- c28382efc978	0a8e24db-eee1-5dc4- a206-2f9c8e9e7022

原始 GPT 順序 fsbl/env/opensbi/uboot/bootfs/rootfs；bootfs 268,435,456 bytes、初始 rootfs 6,442,450,944 bytes。板端 /dev/mmcblkN 編號可能變化，按實物與 UUID 辨識；不能硬認 mmcblk0 永遠是 SD。根分區預期 ext4、可讀寫，/boot 為獨立 bootfs。首開後根分區／檔案系統應使用可用媒體空間，允許 GPT 保留區及 ext4 開銷，不要要求 df 等於標稱容量。

PASS：UUID、來源媒體及分區均對應，無 ext4／I/O 錯誤或唯讀降級，擴容結果合理。partition_changed=false 不代表該服務改過 GPT；保存實際結果，不改寫歷史。不要執行 armbian-install、重新分區或對已掛載檔案系統跑修復。

5. 桌面、USB、網路、藍牙與音訊

CM6-008：HDMI 與 GNOME 桌面

關機時先接已知良好螢幕／HDMI 線再冷開。以一般使用者登入桌面，在**桌面終端機**執行，不用 SSH 的空白工作階段代替：

```
{ printf 'session=%s\n' "$XDG_SESSION_TYPE";
  loginctl show-session "$XDG_SESSION_ID" -p Type -p State -p Remote;
  systemctl --no-pager status display-manager;
  for d in /sys/class/drm/card*-*-HDMI-A-*; do
    test -d "$d" || continue
    printf '%s\n' "$d"; cat "$d/status" "$d/modes"
    sha256sum "$d/edid"
  done
  sudo dmesg --color=never;
} > "$E/CM6-008-desktop.txt" 2>&1
```

記錄實際解析度／更新率；開關五次一般視窗、拖曳、輸入中文、鎖定／解鎖。拍清楚桌面 photos/CM6-008-desktop.jpg，同時測擷取器時另存 photos/CM6-008-capture.png。檢查螢幕與擷取器差異以定位 EDID，不用青色畫面代表桌面。

PASS：GNOME Wayland 可操作、文字與色彩正常、無破圖／當機，EDID 與實際模式一致；若需其他模式須另記配置。**FAIL：**無畫面、青色／黑屏、EDID 失敗或桌面崩潰。無螢幕或登入條件則 BLOCKED。GPU 離屏 PASS 不得代替本項。

CM6-009：USB 枚舉、鍵鼠與資料

逐一依載板外露 USB 埠編號記錄。先存 `lsusb -t`、`lsusb` 與 `sudo dmesg --color=never`，插入鍵鼠後在文字編輯器輸入 CM6-USB-TEST、移動／點擊；每個埠各一次，保存 CM6-009-usb.txt 與 photos/CM6-009-input.jpg。

隨身碟只能用**明確准許寫入的既有檔案系統**，不格式化。用 `findmnt`、`lsblk` 核對型號／序號與掛載點，人工填入 USB_MNT：

```
read -r -p '已核對且准寫的 USB 掛載目錄：' USB_MNT
findmnt -T "$USB_MNT"
lsblk -b -o NAME,MODEL,SERIAL,SIZE,TRAN,MOUNTPOINTS
# 確認 findmnt 的 SOURCE 為該 USB 分區後才繼續。
USB_TMP=$(mktemp -d "$USB_MNT/cm6-test.XXXXXXX") || exit 1
dd if=/dev/urandom of="$USB_TMP/payload.bin" bs=1M count=16 conv=fsync status=none || exit 1
test "$(stat -c %s "$USB_TMP/payload.bin")" = 16777216 || exit 1
sha256sum "$USB_TMP/payload.bin" | tee "$E/CM6-009-write.sha256"
set -o pipefail
# direct 避開檔案頁面快取；檔案系統不支援時標 BLOCKED，另做卸載重掛回讀。
dd if="$USB_TMP/payload.bin" of="$E/CM6-009-readback.bin" \
    bs=1M count=16 iflag=direct,fullblock status=none || exit 1
test "$(stat -c %s "$E/CM6-009-readback.bin")" = 16777216 || exit 1
cmp "$USB_TMP/payload.bin" "$E/CM6-009-readback.bin" || exit 1
sha256sum "$E/CM6-009-readback.bin" > "$E/CM6-009-read.sha256"
rm -- "$USB_TMP/payload.bin"
rmdir -- "$USB_TMP"
sync
```

PASS：每個適用埠能枚舉且鍵鼠可用，資料相同、無新增 USB reset／I/O 錯誤；安全卸載後重插仍可用。僅有 HID 查詢回覆不足。沒有准寫 USB 時資料子項 BLOCKED，不擅自改碟。

CM6-010／011／012：eth0、eth1 與 Wi-Fi 的共同資料測試

每個介面分別執行以下流程；證據前綴分別為 CM6-010-eth0、CM6-011-eth1、CM6-012-wlan0。若實際 Wi-Fi 介面不是 wlan0，按 `ip -br link` 核對並記錄。不以一埠成功推定另一埠。

【主機】在與板子相同的隔離測試 LAN 選定實際 IPv4，開啟只提供 8 MiB 測試資料的 HTTP 對端；程式不讀／寫磁碟，不對公網開放。防火牆只允許測試板存取 TCP 8000。

```
read -r -p '主機的測試 LAN IPv4：' PEER
python3 "$DOC/驗證工具/cm6_rc3_net_peer.py" --bind "$PEER" \
    |& tee "$E/host/CM6-network-peer.txt"
```

【板端】eth0／eth1 各插已知良好線到 GbE 交換器，記錄對應實體 RJ45、LED、拔線後 carrier 消失及再插線恢復。啟用某介面可能中斷 SSH，操作使用 UART／桌面或保持管理用另一埠；不要停掉唯一管理通道。

```
sudo nmcli device connect eth0
sudo ethtool eth0
# eth0 測完後，同樣對 eth1 執行；保存每次命令輸出。
```

Wi-Fi 先記 `nmcli device status`、`iw dev`，使用互動密碼詢問，**不把密碼寫在命令或錄影中**：

```
read -r -p '現場測試 SSID：' TEST_SSID
nmcli device wifi list
```

```
nmcli --ask device wifi connect "$TEST_SSID"
unset TEST_SSID
iw dev wlan0 link
```

記錄頻段、頻率、訊號、AP 型號／韌體、距離與障礙物。至少跑現場要求的 2.4 GHz／5 GHz；沒有該頻段 AP 則該子項 BLOCKED。本輪既有證據只涵蓋 5 GHz 5220 MHz，不能代替 2.4 GHz。

板端每次設定一個介面和新前綴。先在下列三組中選一組，不要三行都貼上：

```
IFACE=eth0; PREFIX=CM6-010-eth0
# IFACE=eth1; PREFIX=CM6-011-eth1
# IFACE=wlan0; PREFIX=CM6-012-wlan0-5G-R01
```

```
export PATH="/usr/sbin:$PATH"
read -r -p '主機測試 LAN IPv4: ' PEER
D="$E/$PREFIX"
mkdir "$D" || exit 1
ip -s -j link show dev "$IFACE" > "$D/before.json"
ip route get "$PEER" oif "$IFACE" > "$D/route.txt"
ping -I "$IFACE" -c 10 -W 2 "$PEER" > "$D/ping.txt" 2>&1
run_curl() {
    local STEP=$1 RC
    shift
    curl --fail --max-time 60 --interface "$IFACE" "$@" 2> "$D/$STEP.stderr"
    RC=$?
    printf '%s\n' "$RC" > "$D/$STEP.exit"
    return "$RC"
}
run_curl expected "http://$PEER:8000/sha256" -o "$D/expected.sha256" || exit 1
run_curl download "http://$PEER:8000/payload" -o "$D/download.bin" || exit 1
stat -c '%s' "$D/download.bin" > "$D/download.bytes"
test "$(cat "$D/download.bytes")" = 8388608 || exit 1
sha256sum "$D/download.bin" | cut -d ' ' -f 1 > "$D/download.sha256"
run_curl upload --data-binary "@$D/download.bin" \
    "http://$PEER:8000/upload" -o "$D/upload.sha256" || exit 1
cmp "$D/expected.sha256" "$D/download.sha256"
printf 'download_compare_rc=%s\n' "$?" | tee "$D/compare.txt"
cmp "$D/expected.sha256" "$D/upload.sha256"
printf 'upload_compare_rc=%s\n' "$?" | tee -a "$D/compare.txt"
ip -s -j link show dev "$IFACE" > "$D/after.json"
{ nmcli -f GENERAL,IP4 device show "$IFACE";
  sudo ethtool "$IFACE"; sudo dmesg --color=never;
} > "$D/link-and-kernel.txt" 2>&1
```

每條 curl 都應退出 0，下載檔必須 **8,388,608 bytes**；缺任一輸出先停，不因兩個空檔 cmp 相同就通過。核對 before.json／after.json 的 stats64（或 stats）收送 bytes 確實增加、errors／dropped 差值；資料雜湊須同時符合對端固定資料 SHA，見工具啟動輸出。固定資料 SHA-256 為 7d212b9c884f5c77896de960ae17cc341cda43b14d6a971f34ca29ebd4badf7f，三份雜湊檔均須逐字等於此值；對端可讀不代表測試路徑正確，仍須檢查介面收送計數。

有線 PASS：指定埠 Link detected yes、1000 Mb/s／Full、DHCP、10 次 ping 零丟包、雙向 SHA 相同、收送錯誤及丟棄零增；拔插後可恢復。已知 100 Mbps 對端不作 GbE PASS，先更換或列 BLOCKED。

Wi-Fi PASS：各要求頻段連線、上述資料／ping 都符合，且收送錯誤與丟棄零增。另做同時長閒置前後計數，保存 \$D/idle-before.json、\$D/idle-after.json；若丟棄增加仍列 FAIL 並註明閒置也增加，不推定測試負載造成。既有兩輪資料完整但每輪 rx_dropped +1 的結果保留，不寫整項 PASS。

CM6-013：藍牙乾淨首開

讀取 CM6-005 留下的 CM6-013-fresh-bluetooth.txt，**在任何 unblock／power on 前確認：**有 hci0、HCI rfkill 的 Soft blocked: no／Hard blocked: no、Powered: yes。另記 sudo journalctl -b -u bluetooth -u bpi-cm6-bluetooth --no-pager 至 CM6-013-services.txt。

PASS：乾淨首開自行到達可用狀態。**FAIL：**控制器缺失、軟／硬封鎖、Powered no、attach／HCI timeout。已修改才開始記錄則 BLOCKED，需乾淨重燒後重測；後續成功不能改寫首開結果。

CM6-014：解除已記錄的封鎖後，開關與掃描

先保存 CM6-013 失敗。只有確認為 hci0 對應的 rfkill ID 才執行 sudo rfkill unblock <該數字ID>；不要 unblock all 或刪持久狀態。將操作與前後狀態寫 CM6-014-unblock.txt。若硬封鎖，先查板端供電／設定，不強制忽略。

```
{ for n in 1 2 3; do
    printf 'cycle=%s\n' "$n"
    timeout 10 bluetoothctl power off
    sleep 2
    timeout 10 bluetoothctl power on
    sleep 2
    timeout 10 bluetoothctl show
done
sleep 35
sudo timeout 10 hcitool -i hci0 cmd 0x04 0x0001
timeout 20 bluetoothctl --timeout 15 scan on
bluetoothctl scan off
bluetoothctl devices
} > "$E/CM6-014-toggle-scan.txt" 2>&1
```

將指定測試設備置於可發現模式，避免「附近剛好沒裝置」誤判。**PASS：**三輪開關、35 秒閒置後 HCI 回應皆成功，HCI 完成狀態為 0，LMP 子版本與 rc3 韌體預期 0x8a4c 相符（回覆末尾 4c 8a 為小端序；不能只找這兩個 byte 而忽略錯誤），掃描找到指定設備。timeout／失敗／未找到已知可發現設備為 FAIL；無可發現設備則掃描 BLOCKED。這不包含配對或音訊。

CM6-015：藍牙配對、傳輸及重新連線

用專用測試設備，先記錄匿名設備代號及類型，不記 PIN。打開 GNOME「設定 → 藍牙」，選該設備；在**未錄製畫面**確認兩端配對碼。成功後記錄 bluetoothctl info <現場設備位址> 至 CM6-015-pair.txt。關閉再開啟該測試設備，確認重新連線。

若設備支援 OBEX，以桌面藍牙傳檔介面雙向送同一份無敏感資料的 1 MiB 測試檔；在兩端各算 SHA-256，保存 CM6-015-transfer.sha256。可用 dd if=/dev/urandom of="\$E/CM6-015-payload.bin" bs=1M count=1 status=none 建檔。工具或 OBEX 端缺少則傳輸 BLOCKED；鍵盤不支援 OBEX 可列該子項 NA，但仍需另備可傳檔裝置才能驗收板載傳輸需求。

PASS：配對、重新連線、雙向 SHA 一致；如測藍牙耳機還須完成 CM6-016 對應路由。只掃描成功不足。測後只移除本次新增的配對，不清除其他人的裝置。

CM6-016：音訊輸出、輸入與路由

【桌面一般使用者】先列實際卡與音訊路由；不能把 HDMI、類比、USB 或藍牙混成同一項 PASS。

```
{ aplay -l; arecord -l; pactl info;
  pactl list short sinks; pactl list short sources;
} > "$E/CM6-016-audio-devices.txt" 2>&1
```

在 GNOME 音效設定選明確的輸出，以低音量開始。戴耳機前先調低音量；逐一測載板實際支援的 HDMI／類比及已配對藍牙。每路使用相同指令，終止碼與左右聲道觀察寫到 CM6-016-audio-observation.md：

```
speaker-test -D default -c 2 -t sine -f 440 -l 1 \
  > "$E/CM6-016-playback.txt" 2>&1
# 先在桌面選定專用測試麥克風；環境不可有私人談話。
arecord -D default -f S16_LE -r 48000 -c 1 -d 5 \
  "$E/CM6-016-capture.wav" 2> "$E/CM6-016-capture.txt"
aplay "$E/CM6-016-capture.wav"
```

若 default 不支援格式，依 arecord -D default --dump-hw-params -d 1 /dev/null 與實際路由另建重測，不把無聲檔當 PASS。每換路由使用新檔名後綴如 -hdmi／-analog／-bt。PASS：左右聲道正確、無明顯雜音／斷續，錄音有可辨識測試音且可回放，無 ALSA underrun／驅動錯誤。無相應實體接口可列 NA 並附 revision 依據；缺外設或未測則 BLOCKED。

6. NVMe：先唯讀，寫入必須另確認

CM6-017／018：128 GB／256 GB 唯讀

不得把磁碟序號當固定名稱。先以 lsblk／nvme list 核對型號、序號、容量、所有分區與掛載點，拍實物標籤（遮住不宜外傳資訊）。既有證據是 Kingston 128,035,676,160 bytes 與 Fanxiang 256,060,514,304 bytes，不表示其他容量已測。

【板端】分兩次人工填 DEV，128 GB 使用 CM6-017，256 GB 使用 CM6-018：

```
read -r -p '已核對的 NVMe 整碟路徑：' DEV
read -r -p '本次測項 ID (CM6-017 或 CM6-018)：' ID
D="$E/$ID"
mkdir "$D" || exit 1
{ lsblk -b -o NAME,PATH,MODEL,SERIAL,SIZE,TYPE,FSTYPE,MOUNTPOINTS;
  sudo nvme list; sudo nvme id-ctrl "$DEV";
} > "$D/identity.txt" 2>&1
sudo nvme smart-log "$DEV" -o json > "$D/smart-before.json"
sudo nvme error-log "$DEV" -o json > "$D/errors-before.json"
```

人工核對這是預定 NVMe 整碟、容量足夠、非系統／活躍資料碟後才繼續。若有其他程序寫入，先由資料管理者停止，否則重讀 SHA 變化無法判讀，列 BLOCKED。以下 dd 只有 if= 指向磁碟，**不得加 of=/dev/...**。

```
BYTES=$(sudo blockdev --getsize64 "$DEV") || exit 1
[[ "$BYTES" =~ ^[0-9]+$ ]] && (( BYTES >= 100663296 )) || exit 1
MIB=$((BYTES / 1048576))
set -o pipefail
for POS in 0 $((MIB / 2)) $((MIB - 32)); do
```

```

for ROUND in 1 2; do
    sudo dd if="$DEV" bs=1M skip="$POS" count=32 \
        iflag=direct,fullblock status=none \
        | sha256sum > "$D/read-{$POS}-{$ROUND}.sha256" || exit 1
done
cmp "$D/read-{$POS}-1.sha256" "$D/read-{$POS}-2.sha256" || exit 1
done
sudo nvme smart-log "$DEV" -o json > "$D/smart-after.json"
sudo nvme error-log "$DEV" -o json > "$D/errors-after.json"
sudo dmesg --color=never > "$D/kernel-after.txt"

```

PASS：三處各 32 MiB 兩讀相同、無短讀／timeout／I/O 錯誤，SMART critical_warning=0，media_errors／num_err_log_entries 不增加。既有非零項另立缺陷，不用前後相同掩蓋健康異常；unsafe_shutdowns 需記前後差值。證據為各自 \$ID/ 全目錄。這不是完整容量讀測，也不是速度基準。

CM6-019：256 GB 既有檔案系統暫存檔讀寫

歷史 128 GB 沒有既有檔案系統，只有唯讀證據；**不得為湊齊項目格式化它**。本項限已確認准寫的 256 GB 既有 ext4，沒有這樣的分區就 BLOCKED。若測其他磁碟，另建測項及授權記錄，不借用本項 PASS。

1. 在 CM6-019-authorization.md 記錄實物序號、分區、所有者、准寫暫存範圍與容量；不得含登入資料。
2. 人工設定分區，先 findmnt --source "\$PART" 確認未掛載；sudo tune2fs -l "\$PART" 應顯示乾淨 ext4。不得測系統 root／boot、未知／隣檔案系統，不自動修復。

```

read -r -p '已核對且准寫的 256 GB NVMe ext4 分區：' PART
lsblk -b -o NAME,PATH,MODEL,SERIAL,SIZE,FSTYPE,MOUNTPOINTS "$PART"
findmnt --source "$PART"
sudo tune2fs -l "$PART" > "$E/CM6-019-ext4-before.txt"
# 僅在未掛載、狀態乾淨、身分正確並獲准後執行以下步驟。
M=$(mktemp -d /tmp/cm6-nvme.XXXXXXXX) || exit 1
sudo mount -t ext4 -o rw,nosuid,nodev,noexec "$PART" "$M" || exit 1
findmnt -T "$M" | tee "$E/CM6-019-mount.txt"
df -B1 "$M"

```

確認可用空間至少 512 MiB，板端證據目錄亦至少 300 MiB；只寫新建隨機目錄。保留前後 SMART（命令同 CM6-018，輸出改 CM6-019-smart-before.json／CM6-019-smart-after.json）。

```

NV_TMP=$(sudo mktemp -d "$M/cm6-test.XXXXXXXX") || exit 1
dd if=/dev/urandom of="$E/CM6-019-reference.bin" \
    bs=1M count=256 conv=fsync status=none || exit 1
test "$(stat -c %s "$E/CM6-019-reference.bin")" = 268435456 || exit 1
sha256sum "$E/CM6-019-reference.bin" | cut -d ' ' -f 1 \
    > "$E/CM6-019-expected.sha256"
sudo dd if="$E/CM6-019-reference.bin" of="$NV_TMP/payload.bin" \
    bs=1M conv=fsync status=none || exit 1
sudo stat -c %s "$NV_TMP/payload.bin" | tee "$E/CM6-019-written.bytes"
test "$(cat "$E/CM6-019-written.bytes")" = 268435456 || exit 1
set -o pipefail
sudo dd if="$NV_TMP/payload.bin" bs=1M count=256 \
    iflag=direct,fullblock status=none | sha256sum | cut -d ' ' -f 1 \
    > "$E/CM6-019-actual.sha256"
READ_RC=$?

```

```
printf 'read_rc=%s\n' "$READ_RC" > "$E/CM6-019-result.txt"
cmp "$E/CM6-019-expected.sha256" "$E/CM6-019-actual.sha256"
printf 'compare_rc=%s\n' "$?" >> "$E/CM6-019-result.txt"
# 只刪本輪新建的檔案及空目錄，不使用 rm -rf。
sudo rm -- "$NV_TMP/payload.bin"
sudo rmdir -- "$NV_TMP"
sync
sudo umount "$M"
rmdir -- "$M"
findmnt --source "$PART" > "$E/CM6-019-cleanup.txt"
```

最後 findmnt 應無輸出、退出 1 表示已無該掛載；此處不是測試失敗。若中途失敗先保留證據，再只清理本輪記錄的 NV_TMP，不要盲目重跑。**PASS**：參考檔與 NVMe 寫後檔皆精確 268,435,456 bytes，read_rc=0、compare_rc=0，256 MiB 寫入／fsync／直接回讀 SHA 相符、SMART 錯誤不增、清理與卸載完成。這不是整碟耐久或斷電保護驗收。

7. GPU 與 AI：必須核對實際執行路徑

所有負載前的共用散熱關卡

先完成 CM6-028 的溫度基線。低於 75°C、cooling state 0、頻率上限未受限才啟動 GPU、AI、相機或壓測；不符則 BLOCKED，改善散熱後另開重測。每個負載旁開一個只讀監測終端，按 CM6-028 的命令記錄；達 90°C 立即 Ctrl+C 停止本輪測試。不要關掉核心保護或強制高頻。

CM6-020：GPU 離屏硬體渲染與像素比對

【板端】工具來源、編譯資訊與 source 在 驗證工具/GPU/。核對工具包 SHA 後，必要時先以 chmod u+x "\$T/GPU/cm6_gpu_probe" 恢復傳檔丟失的執行權限：

```
set -o pipefail
sudo timeout --signal=TERM --kill-after=3 30 \
  "$T/GPU/cm6_gpu_probe" --platform gbm \
  --device /dev/dri/renderD128 --seconds 5 \
  |& tee "$E/CM6-020-gpu.txt"
printf 'rc=%s\n' "$?" | tee "$E/CM6-020-exit.txt"
```

PASS：退出 0；實際 GBM／EGL 及 PVR 函式庫、pvrsrvkm、PowerVR B-Series BXE-2-32 均有證據；兩個 512×512 圖樣各 262,144 個像素，容差 1 下零不符；驅動／韌體狀態正常、錯誤計數前後無增，無軟體 renderer。退出 3 表示證據不完整，列 BLOCKED；1、timeout 或像素不符列 FAIL。

保留擴充 dispatch 警告，不宜稱未測擴充可用。歷史 frames=81184 是繪製迴圈計數，**不是螢幕 FPS**；本項與 CM6-008 桌面獨立。

CM6-021／022：AI CPU 正確性與 SpaceMIT 派發

工具固定四包，包含模型、輸入、獨立參考、runner 與 SHA。不需安裝新 ORT 或在板端產生模型；兩種後端在獨立程序執行。先匯入 onnxruntime 再匯入 spacemit_ort 的順序已固化在 runner；不能反轉。

【板端】各解到新的工作目錄，工具包根目錄 SHA256SUMS 必須先通過：

```
AIW="$HOME/cm6-ai-$CASE"
mkdir "$AIW" || exit 1
```

```
mkdir "$AIW/v2" "$AIW/v3" "$AIW/v4" "$AIW/v5"
tar -xzf "$T/AI/cm6-ai-board-bundle-v2.tar.gz" -C "$AIW/v2"
tar -xzf "$T/AI/cm6-ai-conv-probe-v3.tar.gz" -C "$AIW/v3"
tar -xzf "$T/AI/cm6-ai-fp16-probe-v4.tar.gz" -C "$AIW/v4"
tar -xzf "$T/AI/cm6-ai-qdq-probe-v5.tar.gz" -C "$AIW/v5"
python3 -c 'import numpy; import onnxruntime; import spacemit_ort; print(onnxruntime.__version__)' \
> "$E/CM6-021-import.txt" 2>&1
```

逐個執行，每次先通過散熱關卡。輸出目錄必須不存在；每個程序有模型執行逾時：

```
python3 -I "$AIW/v2/run_cm6_ai_v2.py" \
--output "$E/CM6-021-022-v2" --rounds 3 --timeout 25 \
> "$E/CM6-021-022-v2-console.txt" 2>&1
python3 -I "$AIW/v3/run_cm6_ai_v3.py" \
--output "$E/CM6-021-022-v3" --rounds 1 --timeout 25 \
> "$E/CM6-021-022-v3-console.txt" 2>&1
python3 -I "$AIW/v4/run_cm6_ai_v4.py" \
--output "$E/CM6-021-022-v4" --rounds 1 --timeout 25 \
> "$E/CM6-021-022-v4-console.txt" 2>&1
python3 -I "$AIW/v5/run_cm6_ai_v5.py" \
--output "$E/CM6-021-022-v5" --rounds 1 --timeout 25 \
> "$E/CM6-021-022-v5-console.txt" 2>&1
```

逐份讀 summary.json 與原始 ONNX Runtime JSON profile，保存每個模型／輸入 SHA、參考誤差、計算算子、實際 provider、回退、耗時、載入的 ORT 函式庫。runner 整體非零可能是加速項 FAIL，**CPU 子項需按摘要獨立判斷**，不可一併寫成不可運算。

ID	客觀判定
CM6-021	CPU 後端的 MatMul、QLinearConv、Conv+Relu、QDQ Conv 全部輸入／輪次均符合包內誤差門檻、無 timeout，才 PASS；數值錯誤 FAIL，缺依賴／資料 BLOCKED。
CM6-022	除相同數值正確性外，逐節點 profile 必須有本模型的 SpaceMIT 計算節點 、目標計算沒有回退 CPU，且無 session／載入異常，才可判該模型派發 PASS；只有 provider 名稱不夠。全部 CPU 回退明確 FAIL。

v4 以 provider_options 設 SPACEMIT_EP_FLOAT16_MODE=1，不是只設環境變數；v5 是實際 QDQ 模型。已驗 v2~v5 全部回退 CPU，因此 **AI／NPU 加速尚未通過**。K1 的 IME 是 CPU 矩陣指令擴充，不是獨立 NPU；CPU provider 內部是否用 RVV／IME，本次亦未追蹤。不以供應商 TOPS 或 CPU 正確結果冒充加速驗收，也不推定所有其他模型都不能加速。

8. 兩路 IMX415：原版與候選必須分批

CM6-023：原 rc3 的配置與第一路

原 rc3 映像未整合相機候選。原版 sensor0 可用；sensor2 與 I²C 5 未啟用，官方相機套件也未安裝。不得把下面候選的結果填成原版雙路 PASS。先保存原版證據，再評估候選。

參考官方 V00 原理圖：CN15「MIPI CSI0」對 sensor0／SDK csi1；CN16「MIPI CSI1」對 sensor2／SDK csi3，35-pin、4+4 配置。**實際 revision、感測器及轉接排線不一致或無法核實時，本章 BLOCKED**；切勿只憑連接器形狀插入。關機斷電後裝兩顆 IMX415，確認方向、接點、固定及光線，再上電。

【板端】保存原配置：

```
{ sha256sum /boot/bpi-k1-vendor/k1-x_bpi_cm6-eth0-reset.dtb;
cat /boot/extlinux/extlinux.conf;
sudo fdtget -t s /sys/firmware/fdt /soc/cam_sensor@2 status;
sudo fdtget -t s /sys/firmware/fdt /soc/i2c@d4013800 status;
ls -l /dev/cam_sensor* /dev/video*;
dpkg-query -W k1x-cam k1x-cam-lib;
} > "$E/CM6-023-stock-camera.txt" 2>&1
```

原 DTB SHA 預期 f0b9795fda72868cd2bc9a6db0cf5aa2f15ea494f114a5b42adfc1fa931962c1。原版第二路 status disabled、相機套件查不到，是已知缺口，不是操作成功代表功能通過。

將固定官方 DEB 只解開到獨立目錄，不執行 dpkg -i 或替換系統圖形函式庫：

```
CAMROOT="$HOME/cm6-camera-$CASE"
mkdir "$CAMROOT" || exit 1
dpkg-deb -x "$T/camera/k1x-cam_0.2.34_riscv64.deb" "$CAMROOT"
dpkg-deb -x "$T/camera/k1x-cam-lib_0.1.8_riscv64.deb" "$CAMROOT"
python3 "$T/cm6_rc3_camera_test.py" --runtime "$CAMROOT" \
--config "$T/camera/sensor0-imx415-offscreen-500.json" \
--output "$E/CM6-023-sensor0-stock" --check-only
```

--check-only 只檢查，不建立證據目錄。散熱與原版 DTB 通過確認後，執行第一路：

```
sudo python3 "$T/cm6_rc3_camera_test.py" --runtime "$CAMROOT" \
--config "$T/camera/sensor0-imx415-offscreen-500.json" \
--output "$E/CM6-023-sensor0-stock" \
--seconds 60 --start-temperature 75 --temperature-limit 90 \
> "$E/CM6-023-runner.json" 2>&1
```

runner 拒絕既有 /usr/share/camera_json 或 /tmp 相機輸出，不直接刪除它們；遇到阻擋先保存並查明所有者。它只暫設自己的校正目錄連結，結束後移除，保留影格、stdout／stderr、核心前後與溫度。未經核對的自動偵測配置可能往系統目錄寫檔，本手冊不直接執行。

原版第一路子項 PASS：按後面的共同影格判定符合。**原版雙路需求：FAIL（已知配置缺失）**；第二路串流因前置未啟用標 BLOCKED。CSV 備註分清子項，不用候選覆蓋原版結果。

CM6-024：載入獨立相機候選，保存回復路徑

此步會修改待測板本次開機設定，不改交付 ZIP／IMG。只在已完成原版留證、載板／排線匹配、UART 可用且已有可開機 SD 復原卡時做。**rc3 eMMC 尚無候選實測；eMMC 案件先完成原版整套測試，候選另外建案，不能外推 SD 結果。**

候選檔名 k1-x_bpi_cm6-eth0-camera-candidate.dtb，83,731 bytes，SHA：

```
91bb7a9cde6380e16d9dbc12161212aad768cf145de79dd2f51665366e0065c3
```

它只改八個相機相關屬性，保留 eth0 修正。將 camera-dtb-candidate-manifest.json 留存；其中的離線狀態描述是歷史資訊，不把它改成已發布。

【板端】先備份，檢查目的檔不存在，拒絕覆蓋未知候選：

```
ORIG="$E/CM6-024-extlinux.original.conf"
test ! -e "$ORIG" || exit 1
sudo cat /boot/extlinux/extlinux.conf > "$ORIG"
sha256sum "$ORIG" > "$E/CM6-024-extlinux.original.sha256"
CAND='/boot/bpi-k1-vendor/k1-x_bpi_cm6-eth0-camera-candidate.dtb'
sudo test ! -e "$CAND" || exit 1
sudo install -m 644 "$T/camera/k1-x_bpi_cm6-eth0-camera-candidate.dtb" "$CAND"
sha256sum "$CAND" | tee "$E/CM6-024-candidate.sha256"
```

確認 SHA 與上列相同，**先把 ORIG 與 SHA 複製到主機案件目錄**。使用下列唯檔案轉換，保留此媒體自己的 root UUID，不能拿歷史 SD extlinux 覆蓋 eMMC：

```
python3 - "$ORIG" "$E/CM6-024-extlinux.candidate.conf" <<'PY'
from pathlib import Path
import sys
source = Path(sys.argv[1]).read_text()
a = '    FDT /bpi-k1-vendor/k1-x_bpi_cm6-eth0-reset.dtb'
b = '    FDT /bpi-k1-vendor/k1-x_bpi_cm6-eth0-camera-candidate.dtb'
if source.splitlines().count(a) != 1:
    raise SystemExit('原始 FDT 行不符合，停止並回報；不猜測替換')
with Path(sys.argv[2]).open('x') as target:
    target.write(source.replace(a, b))
PY
diff -u "$ORIG" "$E/CM6-024-extlinux.candidate.conf"
```

diff 必須只有 FDT 這一行，其他內容相同；diff 此時退出 1 是預期。確認後才安裝設定、同步、正常關機：

```
sudo install -m 644 "$E/CM6-024-extlinux.candidate.conf" /boot/extlinux/extlinux.conf
sync
sudo systemctl poweroff
```

UART 見 Power down 才切斷電源，等至少五秒；主機依 CM6-005 開新接收 host/CM6-024-candidate-cold uart.bin，再上電。成功登入後，**先執行第 1 章「每次重開／重新登入」路徑捷徑**，重設 CASE/E/T/CAMR00T/ORIG，不重新 mkdir；執行 test -f "\$ORIG" 及 sha256sum -c "\$E/CM6-024-extlinux.original.sha256" 確認備份可用，再核對核心、媒體 UUID，並確認 live DT：

```
{ sudo fdtget -t s /sys/firmware/fdt /soc/cam_sensor@2 status;
  sudo fdtget -t x /sys/firmware/fdt /soc/cam_sensor@2 twsi-index;
  sudo fdtget -t s /sys/firmware/fdt /soc/i2c@d4013800 status;
  ls -l /dev/cam_sensor*; sudo dmesg --color=never;
} > "$E/CM6-024-live-camera.txt" 2>&1
```

預期依次 okay、5、okay。成功登入後**立即還原下一次啟動設定**，讓候選只留在本次記憶體：

```
sudo install -m 644 "$ORIG" /boot/extlinux/extlinux.conf
cmp "$ORIG" /boot/extlinux/extlinux.conf
```

```
sha256sum /boot/extlinux/extlinux.conf > "$E/CM6-024-next-boot-restored.sha256"
sync
```

PASS：候選 SHA/live DT 正確、冷開可用、下一次設定已逐位元組回復；不能因此判相機 PASS。失敗用 CM6-033 復原，保留首次啟動完整證據。關機／重開後第二路會回原版 disabled，後續相機測試須在這一次候選啟動完成。

在同一候選啟動，先重跑第一路，證據使用 CM6-024-sensor0-candidate/（命令同 CM6-023、只換 --output），再做第二路與雙路。不要沿用原版第一路數據取代候選回歸。

CM6-025：第二路單獨串流

```
sudo python3 "$T/cm6_rc3_camera_test.py" --runtime "$CAMROOT" \
--config "$T/camera/sensor2-imx415-offscreen-500.json" \
--output "$E/CM6-025-sensor2" \
--seconds 60 --start-temperature 75 --temperature-limit 90 \
> "$E/CM6-025-runner.json" 2>&1
```

CM6-026：雙路同時、停止與不重置再開

```
sudo python3 "$T/cm6_rc3_camera_test.py" --runtime "$CAMROOT" \
--config "$T/camera/sensor0-2-imx415-dual-online-500.json" \
--output "$E/CM6-026-dual" \
--seconds 60 --start-temperature 75 --temperature-limit 90 \
> "$E/CM6-026-dual-runner.json" 2>&1
# 不重開機、不重載驅動；降溫後立即驗證第二路可再次開啟。
sudo python3 "$T/cm6_rc3_camera_test.py" --runtime "$CAMROOT" \
--config "$T/camera/sensor2-imx415-offscreen-100.json" \
--output "$E/CM6-026-reopen" \
--seconds 60 --start-temperature 75 --temperature-limit 90 \
> "$E/CM6-026-reopen-runner.json" 2>&1
```

CM6-023～026 共同判讀：每個目錄的 summary.json、stdout.log、stderr.log、kernel-before.txt、kernel-after.txt 全部保留。runner 退出 0 / NEEDS_REVIEW 只代表有結果待審，cam-test 退出 0 也不能單獨當 PASS。

- 真正辨識 IMX415、有本路的串流／回呼、設定停止點 500（重開為 100）；核心主要輸出影格皆正常、軟／硬錯誤 0。歷史約有 502／102 幀，尾端額外幀可以記錄，但不能丟失配置所需的幀數。
- 單路應有 cpp0_output_1920x1080_s1920.nv12（3,110,400 bytes）與對應 raw_output0_3864x2192.raw（11,293,184 bytes）；雙路再加 cpp1_output_1920x1080_s1920.nv12、raw_output1_3864x2192.raw，四個檔皆需 SHA 與回讀一致。這是每路一份 NV12／RAW，不是保存全部 500 幀。
- 無新增 stream off timeout、晚到 IRQ、ISP 錯誤或殘留程序／校正連結，才可判停止子項 PASS。以核心前後差異判定新增警告，不能把前一輪警告混進重開輪。
- 雙路數據皆完整但 pipe1 停止逾時仍列 **CM6-026 FAIL**，即使隨後第二路可重開。歷史正是此情況：兩路約 19.394 FPS、零影格錯誤，但停止等候 500 ms 逾時；重開成功沒有消除缺陷。
- 每個子項保存開始／停止時間與實測 FPS；配置 30 FPS 而實測約 19.4，不可宣稱 30 FPS 通過。功能資料、停止、幀率、畫質分開記錄。

CM6-027：畫質與幀率

在均勻光照下擺放靜態解析度／色卡／灰階圖，讓測試人員可辨識每路場景；記距離、照度（有儀表時）、曝光／增益、鏡頭與對焦位置。若要移動鏡頭按模組規格操作，排線仍禁止帶電插拔。重新執行兩個單路及雙路到新目錄（加-quality-R01），保存原始 NV12／RAW；條件、觀察及幀率寫入 CM6-027-quality.md。

【測試主機】只對已複製的影格轉預覽，原檔不可覆寫。切換到含影格的工作目錄，指定實際單路檔案；兩份預覽完成後複製到主機案件 photos/：

```
ffmpeg -n -f rawvideo -pixel_format nv12 -video_size 1920x1080 \  
-i cpp0_output_1920x1080_s1920.nv12 -frames:v 1 CM6-027-port0-preview.png  
# 第二路對應檔案亦執行一次，輸出改 CM6-027-port1-preview.png。
```

PASS：兩路均有對焦清晰、方向正確、無全黑／灰霧／條紋／偏色／破損的圖像；灰階與色卡可辨識；以實測日誌達到本案指定 30 FPS（需記錄感測器模式與容許範圍，由驗證負責人先定義）。若無事先定義的畫質／幀率規格，量測結果保存但規格驗收 BLOCKED；明顯失焦或未達已定門檻則 FAIL。

歷史預覽只有偏灰與模糊邊界，欠缺合格場景；畫質仍待驗，不得抄成 PASS。GPU 離屏與相機離屏均不代表 HDMI 顯示成功。

9. 溫度、降頻與短壓測

CM6-028：空載與各負載的溫度記錄

散熱裝好後，在桌面空載五分鐘；每秒記錄所有 thermal zone、CPU 上限／目前頻率及 cooling 狀態。以下在獨立終端執行；LABEL 每輪改成 idle、gpu、ai-v2、camera-dual 等，不能覆蓋。

```
LABEL=idle  
set -o pipefail  
while ;; do  
    date -u --iso-8601=seconds  
    for Z in /sys/class/thermal/thermal_zone*; do  
        test -d "$Z" || continue  
        printf '%s ' "$Z"; cat "$Z/type" "$Z/temp"  
    done  
    for C in /sys/class/thermal/cooling_device*; do  
        test -d "$C" || continue  
        printf '%s ' "$C"; cat "$C/type" "$C/cur_state"  
    done  
    for P in /sys/devices/system/cpu/cpufreq/policy*; do  
        test -d "$P" || continue  
        printf '%s ' "$P"  
        cat "$P/cpuinfo_max_freq" "$P/scaling_max_freq" "$P/scaling_cur_freq"  
    done  
    sleep 1  
done | tee "$E/CM6-028-$LABEL.txt"
```

五分鐘後 Ctrl+C 結束空載記錄；負載前另啟新記錄，測試結束再 Ctrl+C。溫度原值通常是毫攝氏，94000 即 94°C。以 scaling_max_freq 降低或 cooling state 增加作降頻證據，不能把空載 scaling_cur_freq 較低直接叫過熱降頻。

PASS：本次要求的空載／各負載全程感測有效、未達 90°C、無熱警告與降頻，上限符合此配置預期。**FAIL**：實測過熱／降頻；缺感測器或散熱條件不明 BLOCKED。原本記錄最高 92~94°C、cooling state 2、上限由 1,600,000 降為 1,228,800 kHz，維持 FAIL；115°C 核心臨界門檻不能當允許持續壓測的依據。

CM6-029：30 秒、兩個 worker 的 CPU 短測

rc3 沒有 stress-ng，不為此升級系統。附帶工具只跑受控 CPU 負載，拒絕不安全輸出路徑，不寫儲存裝置。工具會先檢查溫度低於 75°C、可讀散熱與頻率資訊、無降頻，未滿足則 **不啟動 worker**。

```
python3 "$T/cm6_rc3_short_stress.py" \
  --output "$E/CM6-029-short-stress" --seconds 30 --workers 2 \
  > "$E/CM6-029-console.txt" 2>&1
printf 'rc=%s\n' "$?" > "$E/CM6-029-exit.txt"
```

每 0.5 秒記溫度、頻率／cooling；達 90°C、出現降頻、worker 異常或取樣失敗即停。檢查 summary.json、temperature.csv 與所有 worker 退出碼／清理結果。

PASS：完整 30 秒、退出 0、無熱／頻率／程序異常且全部子程序回收。退出 1 FAIL；2 BLOCKED。此項只代表兩個 worker 的 30 秒短測，不等於全核心、GPU+CPU 混合、長時間穩定或量產燒機通過。

10. 重開、關機與復原

完成相機候選後先確認原 extlinux 已恢復。下列原版回歸不能在不明候選上做。把板端證據同步到主機後，再進行電源操作。

CM6-030：暖重啟一次

1. 【板端】保存 `cat /proc/sys/kernel/random/boot_id` 為 CM6-030-before-boot-id.txt；完成 sync。
2. 【主機】按 CM6-005 先開新 UART 檔 `host/CM6-030-reboot.uart.bin`。
3. 【板端】執行 `sudo systemctl reboot`；主機不得切斷電源。
4. 五分鐘內恢復登入後，先用第 1 章捷徑重設板端路徑，保存新的 `boot_id` 至 CM6-030-after-boot-id.txt，重跑身分／掛載核對及收集器至 CM6-030-after/；各埠 ping、藍牙 Powered 與桌面檢查另存 CM6-030-regression.txt。

PASS：boot_id 改變、來源媒體與 UUID 正確、原版 DTB 恢復、無 panic／新增失敗單元，重開後的已驗基本功能仍可用。藍牙先前解除後恢復，只是持久狀態結果，不改寫 CM6-013 的首開 FAIL。

CM6-031：三次真正冷開機

逐次使用 01、02、03，**不要以 reboot 代替斷電**：

1. 保存當前 `boot_id`、正常 sync 後 `sudo systemctl poweroff`。UART 出現 Power down 才切斷實際主供電，確認電源指示熄滅；排除 USB／UART VCC 反向供電。
2. 記錄斷電與上電 UTC 時間，斷電至少五秒。主機先開 `host/CM6-031-cold-01.uart.bin` 接收才上電；另外兩次只改序號。
3. 每次五分鐘內可登入後，先用第 1 章捷徑重設板端路徑，再保存 CM6-031-cold-01-state.txt：boot_id、身分、UUID、`ip -br address`、兩埠 `ethtool`、指定介面 `ping`、`rftkill list`、`bluetoothctl show`、`systemctl --failed`；桌面照片另存相同序號。
4. 三次均核對原版 `sensor2 disabled`，確認候選未偷偷留在預設開機設定；不要為此要求第二路原版通過。

PASS：三次都有可核對的斷電間隔、完整 UART 與狀態，全部正常啟動且原版行為可重現。任何一輪 FAIL，整項 FAIL；缺供電證據則 BLOCKED。已知藍牙首開缺陷仍獨立保留。

CM6-032：正常關機

【主機】先開 host/CM6-032-poweroff uart.bin。板端保存 findmnt、systemctl --failed，確認測試程序結束、測試檔清理與 NVMe 卸載；執行 sync、sudo systemctl poweroff。

記錄命令到 UART Power down 的秒數、電源指示與裝置狀態至 host/CM6-032-observation.txt。**PASS：**兩分鐘內完成受控關機、無檔案系統／卸載錯誤且不自行重啟；最後才關主供電。若無 Power down，先留 UART、等候並列 FAIL，不把直接拔電寫成正常關機成功。

CM6-033：復原與回退演練

不故意破壞分區、模擬掉電寫入或改壞 DTB。用可開機復原 SD 與已保存設定，驗證實際可回到原版：

- **SD 候選能登入：**依 CM6-024 還原 extlinux；暖重啟後核對原 DTB、UUID、網路，保存 CM6-033-restored-state.txt 與 host/CM6-033-recovery uart.bin。
- **SD 候選不能登入：**保存失敗 UART、正常可行時先關機；確需強制斷電則記事故時間。把該卡接讀卡器，按序號及 SD bootfs UUID 人工辨識第 5 分區。先以 ext4 ro,no-load 唯讀掛載（避免 journal replay 寫入）查看 extlinux/extlinux.conf 並與主機備份比較；確認只有本次已知變更後，卸載並將同一已辨識分區掛成可寫，還原備份，sync、卸載再試。無法確認身分則 BLOCKED，不修其他分區。
- **eMMC 不能啟動：**先拔除無關媒體，用已驗 SD 依實際載板開機設定啟動，保存 lsblk、eMMC 型號／容量與 UUID；只讀檢查 eMMC bootfs。確認可回復且有原備份才處理本次設定差異。若需重燒，回 CM6-004，重新確認 eMMC 可覆寫並留完整 Titan 日誌，不能自動格式化。

板端／主機針對已辨識的 bootfs 分區可依下列模板操作。**BOOTPART 必須由人工輸入與核對，不自動找磁碟；BACKUP 必須是該媒體自己的已驗備份。**

```
read -r -p '已核對的 bootfs 分區：' BOOTPART
read -r -p '該案件 extlinux 原始備份絕對路徑：' BACKUP
lsblk -b -o NAME,PATH,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS "$BOOTPART"
sha256sum "$BACKUP"
R=$(mktemp -d /tmp/cm6-recovery.XXXXXXXX) || exit 1
findmnt --source "$BOOTPART"
# 上行應無掛載；有自動掛載時先保存狀態並處理，未確認不得繼續。
sudo mount -t ext4 -o ro,no-load,nosuid,nodev,noexec "$BOOTPART" "$R" || exit 1
sudo cat "$R/extlinux/extlinux.conf"
sudo umount "$R"
# 確認 UUID、備份 SHA 與差異皆屬本案後才執行寫入回復。
sudo mount -t ext4 -o rw,nosuid,nodev,noexec "$BOOTPART" "$R" || exit 1
sudo install -m 644 "$BACKUP" "$R/extlinux/extlinux.conf"
sudo cmp "$BACKUP" "$R/extlinux/extlinux.conf"
sync
sudo umount "$R"
rmdir -- "$R"
```

PASS：實際完成安全回退或復原媒體啟動演練，回到原版身分、UUID、DTB、基本網路；證據含操作者識別媒體的依據與完整 UART。只有寫好備案但未做演練則 BLOCKED。不要為製造測試而故意斷電損壞系統。

11. 證據封存、缺陷、重測與簽核

CM6-034：回傳前檢查

\$E 若仍有 root 所有子目錄，先按第 1 章只對該案件調整所有權；確認無 Permission denied 後，將板端 \$E **明確這個案件目錄**複製到主機的案件 board/，不要整個複製使用者家目錄或開發主機證據根目錄。主機端保留：

```
CM6-A01-sd-rc3-R01/  
  results.csv  
  host/      ← SHA、媒體辨識、Titan、UART 與時間線  
  board/     ← baseline、各 CM6-ID 日誌、profile、相機原始影格  
  photos/    ← 接線、載板、桌面、工具畫面、畫質照片  
  defects/   ← 每個缺陷與相關的重測關聯  
  signoff.md  
  SHA256SUMS
```

原始紀錄不可剪掉失敗、改字或覆蓋。若誤含憑證，立即限制原始檔權限、按內部規則處理，禁止交付該檔；另建明確標示的去識別副本，記錄刪除範圍與原因，分開算 SHA，不冒充原始資料。密碼／金鑰不得放進缺陷報告或新校驗清單。日誌可能有 MAC、IP、裝置名；按收件者權限處理。

只在已完成敏感資訊人工檢查的案件目錄，使用明確白名單計算校驗（不追蹤符號連結）：

```
cd "$HOME/cm6-evidence/$CASE" || exit 1  
find host board photos defects -type l -print  
# 上行若有連結先查明，不打包外部目標；完成 results.csv/signoff.md 後：  
find host board photos defects -type f -print0 \  
  | sort -z | xargs -0 -r sha256sum > SHA256SUMS  
sha256sum results.csv signoff.md >> SHA256SUMS  
sha256sum -c SHA256SUMS > verification.txt 2>&1
```

verification.txt 記這次核對結果，不把校驗檔自身寫入自身造成循環。再次更改 CSV／簽核需重新產生清單。保留原始內容的受控封存與交付副本各自 SHA，報告中說明哪個是原始證據。

CSV 初始全為 BLOCKED／尚未執行，**沒有預填 PASS**。證據檔名 欄為相對案件根目錄的精確檔名，以分號分隔；多輪可增列帶 -R02 的 ID，不刪初次列。沒有執行就保留 BLOCKED 並填原因。不同媒體不共用同一份結果。

缺陷報告模板

每個問題新增 defects/CM6-缺陷編號.md，填下列欄位，不能只貼「不行」：

```
缺陷編號／關聯測試 ID：  
測試者／日期時區／案件：  
模組、載板型號與 revision／散熱與室溫：  
映像完整檔名、ZIP SHA、工具版本：  
SD 或 eMMC／原版或相機候選／live DTB 證據：  
前置條件與從斷電開始的逐步重現命令：  
預期結果／實際結果／首次錯誤時間：  
出現次數／總測試次數：  
原始證據檔名與 SHA：  
影響範圍、是否可恢復、暫行處理：  
重測版本與新證據位置／仍未涵蓋範圍：  
處理負責人／審核結果：
```

重測規則

- 改映像、核心、DTB、韌體或套件，均記新 SHA／版本；一輪只改可追溯的變因。原版與候選不混寫。
- 藍牙首開修正必須**重燒乾淨媒體**，在任何 unblock 前留證；普通 reboot 不能替代。
- 網路修正重跑 eth0、eth1、Wi-Fi 的獨立流量／計數與三次冷開；PHY 修正不可只看連線圖示。
- AI 修正重跑相同模型與 CPU 對照、實際 profile；只有模型不同的成功不能關閉舊模型缺陷。
- 相機停止修正重跑兩個單路、雙路、無重置再開、畫質及幀率；保留原停止警告，不能增加 timeout 後就忽略。
- 散熱修正記新物料／風扇／環境，重跑相同負載；不要拿不同冷卻條件的耗時直接做效能比較。
- SD 與 eMMC 各自完成燒錄、身分、首開、重啟與冷開；單板樣本不代表所有 RAM／載板版本。

最終簽核清單

在 signoff.md 逐項勾選，填測試者、審查者、日期與待辦負責人：

- ☐ 成品名稱／大小／SHA、工具清單及硬體 revision 完整。
- ☐ 選定媒體實際燒錄、正確 UUID 獨立開機、首次 UART 與時間線齊全。
- ☐ CSV 每列有結果、精確證據、測試者／日期；NA 有依據、BLOCKED 有解除條件。
- ☐ eth0、eth1、Wi-Fi、藍牙首開與後續功能分開；GPU、桌面、AI 各自判定。
- ☐ NVMe 128 GB 唯讀與 256 GB 暫存檔讀寫未混淆；沒有格式化未知磁碟。
- ☐ 兩相機原版／候選分開，停止警告、畫質、幀率與熱限制均有結論。
- ☐ 暖重啟、三輪冷開、關機與復原有實作證據；測試程序／暫存連結／掛載已清理。
- ☐ 原始失敗紀錄保留、敏感資訊未交付、所有交付證據 SHA 通過。
- ☐ 每個未通過硬體關卡都有缺陷／負責人；整體只填「候選部分通過／未通過／待補」，不擅自改成正式發布。

12. 已驗限制對照與來源

截至 2026-09-23，本手冊核對的是下表；它是設計重測的依據，不是替測試人員預填結果。

項目	最新證據結論
成品	SD IMG 與兩 ZIP SHA／結構通過；兩 manifest 仍為 candidate_unverified、hardware_validation=pending。
eth0／eth1	新 rc3 SD 均 1 Gbps、基本雙向通訊通過；不是持續吞吐／所有載板保證。
Wi-Fi	5 GHz 兩輪資料 SHA 正確，rx_dropped 各 +1；閒置也觀察到 +1，嚴格檢查未過。
藍牙	乾淨首開有 hci0 但軟封鎖、Powered no；只解除該 HCI 後三輪開關／閒置命令／掃描通過，後續冷開可用。配對／傳輸／音訊未測。
GPU／桌面	PVR 離屏硬體像素驗證通過；HDMI 青色畫面／EDID 失敗，桌面未過。
AI／NPU	四種模型、五組配置 v2～v5 的 CPU 數值正確；SpaceMIT 測試全部計算回退 CPU，加速未過，IME 執行未證實。

項目	最新證據結論
NVMe	Kingston 128 GB 三段重讀通過、無寫入；Fanxiang 256 GB 三段重讀及既有 ext4 的 256 MiB 暫存檔回讀通過。
相機	原版第一路；候選第二路及雙路有資料。雙路停止逾時、畫質與 30 FPS 未過；之後第二路可再開不消除缺陷。
熱與壓力	曾 92~94°C、cooling state 2、CPU 上限降頻；未做長時間壓力或完成持續效能最佳化驗收。
覆蓋	本輪單一 CM6／SD；新 rc3 Titan eMMC、其他模組／載板及 F3 不由此獲得 PASS。

倉內原始來源按時間與條件分開閱讀：

1. [完整 rc3 SD 重燒後實測](#)：最新結果與 24 份指定證據。
2. [CM6 硬體驗證歷史](#)：既有 SD 上逐項修正，不替代新卡首開。
3. [rc2 連線回報](#)：rc2 eMMC 使用者回報，不當成 rc3。
4. [早期外測指南](#)：格式契約與工具入口；其中舊候選／舊憑證資訊不得抄用到本次交付。

pi-88 證據根目錄：

```
/media/pi/SMCI/bpi/cm6-hardware-validation-20260923/
  rc3-full-sd-reflash-20260923/
    測試摘要.json
    測試證據校驗.json
      network-summary.json
      bluetooth-fresh-default.json
      gpu-gbm-render-validation.json
      ai-board-results-v2/summary.json
      ai-board-results-v3/summary.json
      ai-board-results-v4/summary.json
      ai-board-results-v5/summary.json
      nvme-data-validation.json
      camera-dual-results/summary.json
      camera-reopen-results/summary.json
```

以上是受控歷史證據的定位，**不要把整個目錄打包交付**；其中另有私有存取資料，不屬工具白名單。工具來源與精確 SHA 見**本手冊同目錄**（\$DOC）的 BPI-CM6-rc3-驗證工具清單-20260923.json。本手冊只帶合成模型、測試程式與固定配套，不帶歷史實拍內容或登入資料。

官方參考：[CM6 入門與燒錄](#)、[CM6 V00 原理圖](#)、[固定核心相機停止路徑](#)。官方入口可能調整；載板腳位與 Titan 實際版本仍須現場核對，不能以本手冊替代不同 revision 的原理圖。

13. PDF 產生與文件驗證方式

MD 是操作內容來源，PDF 由同一份 MD 產生。使用 tools/render_cm6_rc3_manual.py、Python Markdown 3.5.2、WeasyPrint 66.0、Noto Sans CJK TC／DejaVu Sans Mono，A4 列印樣式。pi-88 既有 WeasyPrint 入口缺少模組，本輪在 /tmp/cm6-rc3-doc-render 建立隔離環境補齊小型依賴，未修改板端或映像。

在倉根目錄重現（已有相同環境時直接執行最後一行）：

```
python3 -m venv --system-site-packages /tmp/cm6-rc3-doc-render  
/tmp/cm6-rc3-doc-render/bin/pip install weasyprint==66.0  
/tmp/cm6-rc3-doc-render/bin/python tools/render_cm6_rc3_manual.py
```

產生後使用 `pdftinfo`、`pdffonts`、`pdftotext -bbox-layout` 核對頁面／字型／文字邊界，再以 `pdftoppm -png -r 100` **逐頁渲染與目視檢查全部頁面**。本輪實際頁數、檢查結果、命令與交付副本比對另記 BPI-CM6-rc3-文件驗證紀錄-20260923.json，不以只抽看封面代替全頁檢查。文件本機檢查不等於新增實機 PASS。